INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Proyecto: *Gestor de citas médicas*



Materia: Paradigmas de programación

Profesora: Guadalupe Ana Gabriela Martínez Hernández.

Grupo: 3CV4.

Integrantes:

- Delgado Villagrana Aurora
- Gutiérrez Sandoval Melyssa Gabriela
- Navarrete Hernández Laura Itzel

Índice

Roles/Usuarios 3
Requerimientos 3
Casos de uso 5
Elección de lenguaje 12
Implementación funcional 14

Roles/Usuarios

- **Recepcionista:** Personal administrativo responsable del control operativo de la agenda médica. Es responsable de gestionar, organizar y monitorear las citas médicas programadas para los doctores, asegurando la correcta asignación de tiempos y la atención.
- **Doctores:** Personal de salud encargado de la consulta y el seguimiento clínico. Es responsable de atender directamente a los pacientes, registrar sus diagnósticos, revisar los motivos de consulta y actualizar el historial clínico.
- **Pacientes:** Usuario beneficiario del servicio médico del hospital. Es responsable de autogestionar sus consultas mediante el sistema, lo que incluye agendar nuevas citas con los doctores disponibles o cancelar aquellas que no pueda asistir.
- **Administrador de área:** Personal de supervisión encargado de registrar, dar de alta y organizar el catálogo de doctores y recepcionistas en la plataforma según el área de especialidad.
- **Director general (Superadmin):** Máxima autoridad administrativa y técnica dentro de la jerarquía del hospital. Es responsable de supervisar la estructura global del sistema y asignar las funciones y responsabilidades a cada Administrador de área.

Requerimientos

Usuario	**Requerimientos**
Paciente	• Puede crear su propia cuenta en el sistema proporcionando sus datos personales y de contacto. • Puede consultar los días y horarios disponibles de los doctores según su especialidad. • Puede agendar una cita médica dentro de los horarios disponibles. • Puede cancelar una cita con 48hrs de anticipación.

	• No puede agendar una cita con menos de 48hrs de anticipación. • No puede reagendar citas, solo cancelar y solicitar una nueva. • Puede visualizar su historial de citas médicas. • Puede visualizar sus datos personales. • No puede visualizar sus recetas médicas. • No puede agendar citas con fechas pasadas.
Doctor	• No puede crear su propia cuenta en el sistema. • No puede editar sus datos personales. • Puede visualizar las citas pendientes. • Puede visualizar el historial médico de sus pacientes. • Puede registrar recetas médicas en el sistema. • Puede cancelar citas.
Recepcionista	• No puede crear su propia cuenta en el sistema. • No puede visualizar las recetas médicas. • No puede visualizar el historial médico de un paciente. • Puede cancelar citas, ya sea por parte del doctor o del paciente. • Puede dar de alta pacientes. • Puede consultar recepcionistas, doctores y pacientes. • Puede registrar un pago (efectivo)
Administrador de área	• No puede crear su propia cuenta en el sistema. • Puede registrar a doctores, pacientes y recepcionistas del área de especialidad.
Director general	• Puede crear su propia cuenta en el sistema. • Puede registrar a los administradores de área.

Casos de uso

- **Registro de paciente:**
    - *Actor principal:* Paciente.
    - *Precondición:* El usuario no debe estar registrado en el sistema ni contar con una sesión activa.
    - *Flujo principal:*
        1. El paciente accede al módulo de registro en la plataforma web.
        2. El sistema despliega el formulario de registro solicitando datos personales (nombre, CURP, teléfono, correo electrónico) y contraseña.
        3. El paciente ingresa la información requerida y envía el formulario.
        4. El sistema valida que los campos no estén vacíos y que el correo/CURP no se encuentren registrados previamente.
        5. El sistema registra las credenciales y perfil del paciente en la base de datos MySQL con rol de **Paciente**.
        6. El sistema confirma el registro exitoso y permite el inicio de sesión.
    - *Regla de negocio***:** Único usuario del sistema con auto-registro permitido.
- **Paciente agenda cita:**
    - *Actor principal:* Paciente.
    - *Precondición*: El paciente debe estar autenticado en el sistema.
    - *Flujo principal:*
        1. El paciente selecciona la especialidad médica deseada.
        2. El sistema despliega el catálogo de doctores disponibles para esa especialidad.
        3. El paciente selecciona un doctor y consulta sus días y horarios disponibles.
        4. El paciente selecciona una fecha y hora propuesta para su consulta.
        5. El sistema ejecuta la validación funcional.
            - Verifica que la fecha sea superior a 48 horas en el futuro respecto al momento actual.
            - Verifica que el doctor no tenga otra cita reservada en ese mismo bloque horaria.

6. Si las validaciones son exitosas, el sistema inserta el registro con estado **AGENDADAPENDIENTE DE PAGO**.
7. El sistema notifica al paciente la confirmación de su cita.

- *Regla de negocio*: No se permiten citas en fechas pasadas ni con menos de 48 horas de anticipación.

- **Registro de recepcionista:**
  - *Actor principal:* Administrador de Área.
  - *Precondición*: El Administrador de Área debe estar autenticado y contar con permisos administrativos.
  - Flujo principal:
    1. El Administrador de Área accede al módulo de gestión de personal.
    2. El sistema muestra el formulario para registrar un recepcionista.
    3. El Administrador de Área ingresa los datos personales y de acceso del recepcionista.
    4. El sistema valida que los campos obligatorios estén completos y que los datos de identificación no estén registrados previamente.
    5. El sistema registra la información del doctor en la base de datos MySQL y le asigna el rol de **Recepcionista**.
    6. El sistema confirma el registro exitoso del recepcionista.
  - *Regla de negocio*: Los recepcionistas no pueden crear su propia cuenta. Su registro debe realizarse mediante un usuario autorizado.
- **Paciente cancela cita:**
  - *Actor principal:* Paciente.
  - *Precondición:* El paciente debe estar autenticado y contar con una cita en estado **AGENDADA/PENDIENTE DE PAGO**.
  - *Flujo principal:*
    1. El paciente accede al historial de sus citas.
    2. El sistema muestra las citas que tiene registradas.
    3. El paciente selecciona la cita que desea cancelar.
    4. El sistema verifica que la cita pueda ser cancelada y que falten al menos 48 horas para su realización.

5. El paciente confirma la cancelación.
6. El sistema actualiza el estado de la cita a **CANCELADA**.
7. El sistema confirma al paciente que la cita fue cancelada

- *Regla de negocio***:** El paciente únicamente puede cancelar una cita con al menos 48 horas de anticipación.

- **Recepcionista agenda cita:**
  - *Actor principal:* Recepcionista.
  - *Precondición:* El recepcionista debe estar autenticado y el paciente debe estar registrado en el sistema.
  - *Flujo principal:*
    1. El recepcionista accede al módulo de agenda.
    2. El sistema solicita identificar al paciente.
    3. El recepcionista selecciona al paciente y la especialidad médica requerida.
    4. El sistema muestra los doctores disponibles de dicha especialidad.
    5. El recepcionista selecciona un doctor y consulta sus horarios disponibles.
    6. El recepcionista selecciona la fecha y hora de la cita.
    7. El sistema verifica que la cita tenga al menos 48 horas de anticipación.
    8. El sistema verifica que el horario seleccionado se encuentre disponible y que no exista otra cita para el doctor en el mismo horario.
    9. Si las validaciones son exitosas, el sistema registra la cita con estado **AGENDADA/PENDIENTE DE PAGO**.
    10. El sistema muestra la confirmación de la cita.
  - *Regla de negocio***:** No se permiten citas en fechas pasadas ni con menos de 48 horas de anticipación.

- **Recepcionista registra paciente:**
  - *Actor principal:* Recepcionista.
  - *Precondición:* El recepcionista debe estar autenticado.
  - *Flujo principal:*
    1. El recepcionista accede al módulo de registro de pacientes.
    2. El sistema muestra el formulario de registro.
    3. El recepcionista captura los datos personales y de contacto del paciente.

4. El sistema valida que los campos obligatorios estén completos.
5. El sistema verifica que el paciente no se encuentre registrado previamente mediante sus datos de identificación.
6. El sistema registra al paciente en la base de datos con rol de **Paciente**.
7. El sistema confirma el registro exitoso.

- *Regla de negocio***:** El registro de pacientes realizado por el recepcionista debe respetar las mismas validaciones de integridad que el auto-registro.

- **Recepcionista procesa pago de cita:**
  - *Actor principal:* Recepcionista.
  - *Precondición:* El recepcionista debe estar autenticado y existir una cita en estado **AGENDADA/PENDIENTE DE PAGO**.
  - *Flujo principal:*
    1. El recepcionista accede al módulo de pagos.
    2. El sistema solicita identificar la cita o al paciente.
    3. El recepcionista selecciona la cita correspondiente.
    4. El sistema muestra la información de la cita y el monto a pagar.
    5. El recepcionista registra el pago realizado en **efectivo**.
    6. El sistema valida que la cita se encuentre pendiente de pago.
    7. El sistema registra el pago.
    8. El sistema actualiza el estado de la cita a **PAGADA**.
    9. El sistema muestra la confirmación del pago.
  - *Regla de negocio***:** Solo se pueden registrar pagos para citas válidas que se encuentren pendientes de pago.
- **Doctor consulta expediente del paciente:**
  - *Actor principal:* Doctor.
  - *Precondición:* El doctor debe estar autenticado y contar con una cita registrada con el paciente.
  - *Flujo principal:*
    1. El doctor accede al módulo de citas pendientes.
    2. El sistema muestra las citas correspondientes al doctor.
    3. El doctor selecciona al paciente de la cita.

4. El sistema consulta el expediente médico asociado al paciente.
5. El sistema muestra al doctor el historial médico disponible.
6. El doctor revisa la información necesaria para la consulta.

- *Regla de negocio***:** El expediente médico únicamente puede ser consultado por personal médico autorizado.

- **Doctor realiza consulta:**
  - *Actor principal:* Doctor.
  - *Precondición:* El doctor debe estar autenticado y contar con una cita válida con el paciente.
  - *Flujo principal:*
    1. El doctor accede a sus citas pendientes.
    2. El doctor selecciona la cita correspondiente.
    3. El sistema muestra los datos del paciente y su información clínica disponible.
    4. El doctor registra los datos correspondientes a la consulta, incluyendo motivo de consulta, diagnóstico y observaciones.
    5. El doctor guarda la información registrada.
    6. El sistema almacena la información en el historial médico del paciente.
    7. El sistema actualiza el estado de la cita a **REALIZADA.**
    8. El sistema confirma el registro de la consulta.
  - *Regla de negocio***:** Solo el doctor responsable de la consulta puede registrar la información clínica correspondiente.
- **Doctor entrega receta médica:**
  - *Actor principal:* Doctor.
  - *Precondición:* El doctor debe estar autenticado y debe existir una consulta válida asociada al paciente.
  - *Flujo principal:*
    1. El doctor accede a la consulta del paciente.
    2. El sistema muestra la opción para registrar una receta médica.
    3. El doctor captura los medicamentos, dosis, indicaciones y demás información correspondiente.

4. El sistema valida que los datos obligatorios estén completos.
5. El doctor confirma la receta.
6. El sistema almacena la receta asociada al expediente médico del paciente.
7. El sistema confirma al doctor el registro exitoso**.**

- *Regla de negocio***:** Las recetas médicas únicamente pueden ser registradas por doctores autorizados.

- **Doctor cancela cita:**
  - *Actor principal:* Doctor.
  - *Precondición:* El doctor debe estar autenticado y debe existir una cita pendiente asociada a él.
  - *Flujo principal:*
    1. El doctor accede a su agenda de citas.
    2. El sistema muestra sus citas pendientes.
    3. El doctor selecciona la cita que desea cancelar.
    4. El sistema solicita la confirmación de la cancelación.
    5. El doctor confirma la cancelación.
    6. El sistema actualiza el estado de la cita a **CANCELADA**.
    7. El sistema registra la cancelación**.**
    8. El sistema confirma la operación.
  - *Regla de negocio***:** El doctor puede cancelar una cita cuando exista una causa que impida realizar la consulta.
- **Director General registra Administrador de Área:**
  - *Actor principal:* Director General.
  - *Precondición*: El Director General debe estar autenticado y contar con permisos de administración global.
  - Flujo principal:
    1. El Director General accede al módulo de administración de usuarios.
    2. El sistema muestra la opción para registrar un Administrador de Área.
    3. El Director General ingresa los datos personales y de acceso del administrador.
    4. El Director General asigna el área de especialidad que administrará.

5. El sistema valida que los datos sean correctos y que el usuario no esté registrado previamente.
6. El sistema registra al usuario con el rol de **Administrador de Área**.
7. El sistema asocia al administrador con el área de especialidad seleccionada.
8. El sistema confirma el registro exitoso.

- *Regla de negocio*: Solo el Director General puede registrar Administradores de Área.

- **Autenticación e Inicio de Sesión:**
  - *Actor principal:* Paciente, Doctor, Recepcionista, Administrador de Área y Director General.
  - *Precondición:* El usuario debe encontrarse previamente registrado en el sistema.
  - *Flujo principal:*
    1. El usuario accede al módulo de inicio de sesión.
    2. El sistema solicita correo electrónico y contraseña.
    3. El usuario proporciona sus credenciales.
    4. El sistema valida las credenciales contra la información almacenada.
    5. Si las credenciales son correctas, el sistema identifica el rol del usuario.
    6. El sistema crea una sesión autenticada.
    7. El sistema redirige al usuario al menú correspondiente a sus permisos**.**
  - *Flujo alternativo:*
    1. Si las credenciales son incorrectas, el sistema informa que los datos proporcionados no son válidos y permite intentar nuevamente.
  - *Regla de negocio***:** El usuario únicamente podrá acceder a las funcionalidades correspondientes a su rol.
- **Cierre de sesión:**
  - *Actor principal:* Paciente, Doctor, Recepcionista, Administrador de Área y Director General.
  - *Precondición:* El usuario debe tener una sesión activa.
  - *Flujo principal:*

1. El usuario selecciona la opción **Cerrar sesión**.
2. El sistema invalida la sesión activa.
3. El sistema elimina o invalida las credenciales temporales asociadas a la sesión.
4. El sistema redirige al usuario a la pantalla de inicio de sesión.
5. El sistema confirma el cierre de sesión.

- *Regla de negocio*: Después de cerrar sesión, el usuario deberá autenticarse nuevamente para acceder a las funcionalidades protegidas.

- **Administrador de Área registra a Doctor:**
  - *Actor principal:* Administrador de Área.
  - *Precondición:* El Administrador de Área debe estar autenticado y debe tener asignada el área de especialidad correspondiente.
  - *Flujo principal:*
    1. El Administrador de Área accede al módulo de gestión de doctores.
    2. El sistema muestra el formulario de registro.
    3. El administrador ingresa los datos personales, profesionales y de contacto del doctor.
    4. El sistema valida que los campos obligatorios estén completos.
    5. El sistema verifica que el doctor no esté registrado previamente.
    6. El sistema registra al doctor y le asigna el rol de **Doctor**.
    7. El sistema asocia al doctor con el área de especialidad del administrador.
    8. El sistema confirma el registro exitoso
  - *Regla de negocio***:** El Administrador de Área únicamente puede registrar doctores pertenecientes a su área de especialidad.

Estado de cita: Agendada y pendiente de pago, cancelada, realizada y pagada

Elección de lenguaje

-Backend: Node.js con JavaScript

Para el backend se seleccionó el entorno de ejecución Node.js sobre JavaScript, enfocándose en multiparadigma (imperativo, funcional y orientado a eventos). Su arquitectura se basa en

un bucle de eventos y operaciones de entrada/salida, resulta excelente para el dominio de un gestor de citas médicas, donde la mayor parte de la carga computacional consiste en transacciones de red y base de datos. Asimismo, compartir JavaScript en ambos extremos del sistema simplifica la serialización de datos nativa en JSON, optimiza la curva de desarrollo y reduce errores semánticos entre capas.

-Frontend: JavaScript (AngularJS), HTML5 y CSS3

Para la capa del cliente se implementó la combinación de **HTML5**, **CSS3** y **JavaScript** bajo el framework **AngularJS**, fundamentada en la convergencia de paradigmas de programación:

- **Paradigma Declarativo (HTML5 y CSS3): HTML5** define la estructura y semántica de las vistas mediante plantillas declarativas; junto con las directivas de AngularJS, describe el *qué* debe mostrarse sin manipular el DOM. **CSS3** aplica un modelo declarativo basado en reglas para desacoplar por completo la presentación visual del sistema.
- **Paradigma Arquitectónico MVC / MVVM:** Separa estrictamente la interfaz de usuario (Vista en HTML/CSS) de la lógica de negocio y estado (Controladores y Servicios en JS), asegurando alta cohesión y bajo acoplamiento.
- **Paradigma Multiparadigma y Reactivo (JavaScript con AngularJS):**
    - **Funcional:** Uso de funciones de primera clase, callbacks y métodos de orden superior para procesar catálogos y transformar colecciones de datos médicos.
    - **Orientado a Objetos basado en prototipos:** Modela entidades y servicios compartidos.
    - **Reactivo / Enlace de datos:** Sincroniza automáticamente el modelo y la vista en tiempo real, agilizando la actualización interactiva de horarios y disponibilidad clínica.
    - **Inyección de dependencias:** Fomenta la modularidad e inversión de control al suministrar componentes y servicios HTTP sin instanciación rígida.

-Base de datos: MySQL (Motor Relacional)

Se seleccionó el sistema gestor de bases de datos relacional **MySQL**. La elección se fundamenta en el soporte nativo para restricciones de integridad referencial (*Foreign Keys*), bloqueos a nivel de fila e índices compuestos, ya que permite:

Evitar conflictos de agenda: Prevenir empalmes de horarios y citas duplicadas mediante restricciones UNIQUE y transacciones atómicas al momento en que un paciente o recepcionista reserva un espacio.

Modelado jerárquico sólido: Establecer con precisión la estructura de roles y permisos (Director general, Administrador de área, Recepcionista, Doctor y Paciente) asegurando la persistencia estricta de historiales clínicos y recetas.

Implementación funcional

1. Caso de uso elegido: Paciente agenda cita

1.1 Describir la funcionalidad.

¿Qué resuelve? Valida la solicitud de un paciente para agendar una cita médica, aplicando las reglas antes de confirmarla.

¿Quién la usa? Paciente y recepcionista.

¿Qué datos recibe? Datos de solicitud (ID paciente, ID doctor y horario de cita), horario laboral del doctor, citas ocupadas y fecha/hora actual.

¿Cómo transforma los datos? Aplica cadena de validación pura y si todas se cumplen, construye la nueva cita para agendar (Fecha no pasada, anticipación mínima de 48hrs)

¿Qué devuelve? La cita construida o lista de motivos de rechazo de la cita

1.2. Solución Funcional (Pseudo-código)

La solución se modela como una cadena de funciones puras: cada regla recibe la solicitud y el contexto (incluida la fecha/hora actual) y devuelve una lista de motivos de rechazo, sin modificar datos ni consultar la base de datos. La función principal combina los resultados de todas las reglas; si la lista queda vacía, construye la cita.

```
CONSTANTES
    HORAS_MIN_ANTICIPACION ← 48
    DURACION_CITA_MIN      ← 30
    ESTADO_INICIAL         ← "AGENDADA/PENDIENTE DE PAGO"
```

```
REGLAS ← [ reglaFechaNoPasada, reglaAnticipacion,
           reglaHorarioLaboral, reglaSinEmpalme ]

FUNCIÓN validarCampos(solicitud) → lista de motivos
    RETORNA un motivo por cada dato faltante o inválido
            entre idPaciente, idDoctor e inicio

FUNCIÓN reglaFechaNoPasada(solicitud, contexto) → lista de motivos
    SI solicitud.inicio < contexto.ahora ENTONCES
        RETORNA ["La fecha de la cita no puede ser pasada"]
    RETORNA []

FUNCIÓN reglaAnticipacion(solicitud, contexto) → lista de motivos
    horas ← (solicitud.inicio − contexto.ahora) expresado en horas
    SI horas ≥ 0 Y horas < HORAS_MIN_ANTICIPACION ENTONCES
        RETORNA ["La cita requiere al menos 48 horas de anticipación"]
    RETORNA []

FUNCIÓN reglaHorarioLaboral(solicitud, contexto) → lista de motivos
    dentro ← EXISTE bloque EN contexto.horarioDoctor TAL QUE
                bloque.dia = díaDeLaSemana(solicitud.inicio)
                Y solicitud.inicio ≥ bloque.desde
                Y solicitud.inicio + DURACION_CITA_MIN ≤ bloque.hasta
    SI NO dentro ENTONCES
        RETORNA ["El horario está fuera del horario laboral del doctor"]
    RETORNA []

FUNCIÓN reglaSinEmpalme(solicitud, contexto) → lista de motivos
    delDoctor ← FILTRAR contexto.citasOcupadas DONDE
                    cita.idDoctor = solicitud.idDoctor Y cita.estado ≠ "CANCELADA"
    SI EXISTE cita EN delDoctor QUE se traslape con solicitud ENTONCES
        RETORNA ["El doctor ya tiene una cita en ese horario"]
    RETORNA []

FUNCIÓN agendarCita(solicitud, contexto) → resultado
    errores ← validarCampos(solicitud)
    SI errores NO está vacía ENTONCES
        RETORNA Rechazo(errores)

    motivos ← APLANAR( MAPEAR REGLAS CON regla → regla(solicitud, contexto) )

    SI motivos está vacía ENTONCES
        RETORNA Aceptada( nuevaCita(solicitud, ESTADO_INICIAL) )
    SI NO
        RETORNA Rechazo(motivos)
```

1.3. Implementación del lenguaje.

Se implementó en JavaScript (Node.js), el lenguaje elegido para el backend. Elementos del paradigma funcional aplicados: funciones puras, funciones de orden superior (filter, some, flatMap), composición de reglas mediante una lista de funciones (REGLAS), inmutabilidad (Object.freeze, sin modificar las entradas) y fecha actual recibida como parámetro en lugar de leer el reloj dentro de la función.

```
'use strict';

// ---------- Constantes ----------
const HORAS_MIN_ANTICIPACION = 48;
const DURACION_CITA_MIN = 30;
const ESTADO_INICIAL = 'AGENDADA/PENDIENTE DE PAGO';
const MS_POR_MIN = 60 * 1000;
const MS_POR_HORA = 60 * MS_POR_MIN;

// ---------- Funciones auxiliares puras ----------
const aMinutos = (hhmm) => {
  const [h, m] = hhmm.split(':').map(Number);
  return h * 60 + m;
};
const minutosDelDia = (fecha) => fecha.getHours() * 60 + fecha.getMinutes();
const finDeCita = (inicio) => new Date(inicio.getTime() + DURACION_CITA_MIN *
MS_POR_MIN);
const seTraslapan = (a, b) => a < finDeCita(b) && b < finDeCita(a);
const horasDeAnticipacion = (inicio, ahora) => (inicio.getTime() - ahora.getTime()) /
MS_POR_HORA;

// ---------- Reglas: (solicitud, contexto) => lista de motivos ----------
const validarCampos = (s) => [
  ...(s.idPaciente == null ? ['Falta el ID del paciente'] : []),
  ...(s.idDoctor == null ? ['Falta el ID del doctor'] : []),
  ...(!(s.inicio instanceof Date) || Number.isNaN(s.inicio.getTime())
    ? ['Falta la fecha y hora de la cita o no es válida']
    : []),
];

const reglaFechaNoPasada = (s, { ahora }) =>
  horasDeAnticipacion(s.inicio, ahora) < 0 ? ['La fecha de la cita no puede ser
pasada'] : [];

const reglaAnticipacion = (s, { ahora }) => {
  const horas = horasDeAnticipacion(s.inicio, ahora);
  return horas >= 0 && horas < HORAS_MIN_ANTICIPACION
    ? [`La cita requiere al menos ${HORAS_MIN_ANTICIPACION} horas de anticipación`]
    : [];
};

const reglaHorarioLaboral = (s, { horarioDoctor }) => {
  const inicio = minutosDelDia(s.inicio);
  const dentro = horarioDoctor.some(
    (b) =>
      b.dia === s.inicio.getDay() &&
      inicio >= aMinutos(b.desde) &&
      inicio + DURACION_CITA_MIN <= aMinutos(b.hasta)
  );
  return dentro
    ? []
    : ['El horario solicitado está fuera del horario laboral del doctor'];
};

const reglaSinEmpalme = (s, { citasOcupadas }) =>
  citasOcupadas
    .filter((c) => c.idDoctor === s.idDoctor && c.estado !== 'CANCELADA')
    .some((c) => seTraslapan(s.inicio, c.inicio))
    ? ['El doctor ya tiene una cita en ese horario']
    : [];

const REGLAS = [
  reglaFechaNoPasada,
  reglaAnticipacion,
  reglaHorarioLaboral,
  reglaSinEmpalme,
];
```

```
// ---------- Constructores de resultado (inmutables) ----------
const construirCita = (s) =>
  Object.freeze({
    idPaciente: s.idPaciente,
    idDoctor: s.idDoctor,
    inicio: new Date(s.inicio.getTime()),
    duracionMin: DURACION_CITA_MIN,
    estado: ESTADO_INICIAL,
  });
const aceptar = (cita) => Object.freeze({ ok: true, cita });
const rechazar = (motivos) => Object.freeze({ ok: false, motivos:
Object.freeze(motivos) });

// ---------- Función principal (pura: sin efectos secundarios) ----------
const agendarCita = (solicitud, contexto) => {
  const errores = validarCampos(solicitud);
  if (errores.length > 0) return rechazar(errores);

  const motivos = REGLAS.flatMap((regla) => regla(solicitud, contexto));
  return motivos.length === 0 ? aceptar(construirCita(solicitud)) : rechazar(motivos);
};

module.exports = { agendarCita, ESTADO_INICIAL };
```

Ejecución de banco de pruebas.

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
PF-01	Cita válida	Dr. 2, jue 08-oct 10:00	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	**Aprobado**
PF-02	Fecha pasada	Dr. 2, vie 02-oct 10:00	Rechazada: La fecha de la cita no puede ser pasada	Rechazada: La fecha de la cita no puede ser pasada	**Aprobado**
PF-03	Menos de 48 h (24 h)	Dr. 2, mar 06-oct 10:00	Rechazada: La cita requiere al menos 48 horas de anticipación	Rechazada: La cita requiere al menos 48 horas de anticipación	**Aprobado**
PF-04	Límite exacto de 48 h	Dr. 2, mié 07-oct 10:00	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	**Aprobado**
PF-05	Límite: 47 h 30 min	Dr. 2, mié 07-oct 09:30	Rechazada: La cita requiere al menos 48 horas de anticipación	Rechazada: La cita requiere al menos 48 horas de anticipación	**Aprobado**
PF-06	Horario ya ocupado	Dr. 2, jue 08-oct 11:00 (cita agendada)	Rechazada: El doctor ya tiene una cita en ese horario	Rechazada: El doctor ya tiene una cita en ese horario	**Aprobado**
PF-07	Horario con cita CANCELADA	Dr. 2, jue 08-oct 12:00	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	**Aprobado**

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
PF-08	Mismo horario, otro doctor ocupado	Dr. 2, jue 08-oct 13:00 (ocupado el Dr. 3)	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	**Aprobado**
PF-09	Fuera de horario laboral	Dr. 2, jue 08-oct 16:00	Rechazada: El horario solicitado está fuera del horario laboral del doctor	Rechazada: El horario solicitado está fuera del horario laboral del doctor	**Aprobado**
PF-10	Día no laborable	Dr. 2, sáb 10-oct 10:00	Rechazada: El horario solicitado está fuera del horario laboral del doctor	Rechazada: El horario solicitado está fuera del horario laboral del doctor	**Aprobado**
PF-11	Datos incompletos	Sin ID de doctor, jue 08-oct 10:00	Rechazada: Falta el ID del doctor	Rechazada: Falta el ID del doctor	**Aprobado**
PF-12	Varios motivos a la vez	Dr. 2, mar 06-oct 18:00	Rechazada: La cita requiere al menos 48 horas de anticipación | El horario solicitado está fuera del horario laboral del doctor	Rechazada: La cita requiere al menos 48 horas de anticipación | El horario solicitado está fuera del horario laboral del doctor	**Aprobado**
PF-13	Pureza: entradas congeladas y resultado repetible	Mismo caso que PF-01 con Object.freeze en todas las entradas	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	Aceptada (estado AGENDADA/PENDIENTE DE PAGO)	**Aprobado**

Contexto de las pruebas: fecha/hora actual simulada = lunes 05-oct-2026 10:00; horario del Dr. 2 = lunes a viernes de 09:00 a 15:00; citas de 30 minutos; citas registradas: Dr. 2 jue 08-oct 11:00 (AGENDADA/PENDIENTE DE PAGO) y 12:00 (CANCELADA), Dr. 3 jue 08-oct 13:00 (PAGADA).

La anticipación mínima de 48 h se toma como "al menos 48 h", por lo que exactamente 48 h sí se acepta.

Resultado de la ejecución (node pruebas_agendar_cita.js): 13 de 13 pruebas aprobadas.